

CAP 4105 — Introduction to Machine Learning

Project 2: Tree-Based Methods & Kernel Methods

Topics: Decision Trees · Random Forest · Support Vector Machines · Model Comparison

Project Overview

In Project 1, you built linear models (Linear Regression, Logistic Regression) for student performance prediction. In this project, you will discover that **linear models have fundamental limitations** when the relationship between features and outcomes is nonlinear.

You will work with a new version of the student performance dataset that contains **nonlinear patterns** — for example, a "burnout effect" where studying too many hours actually hurts performance. You will implement a Decision Tree classifier **from scratch using NumPy**, then use **scikit-learn** to train SVMs and Random Forests, and finally compare all models to understand when and why each method works.

This project uses the **same features** as Project 1 (`hours_studied`, `sleep_hours`, `attendance_rate`, `prev_exam_score`) but with a new data distribution where nonlinear methods should significantly outperform linear ones.

Logistics

Item	Detail
Team Size	1–3 students (individual or group)
Submission	Upload <code>.ipynb</code> file directly to Canvas
Deliverables	Jupyter Notebook (<code>.ipynb</code>) with all code, outputs, and analysis
Due Date	March 29, 2026, 11:59 PM

Submission Requirements

1. **You must submit a Jupyter Notebook (`.ipynb`) file directly to Canvas.** Python scripts (`.py`), PDFs, or other formats will not be accepted.
2. **Your notebook must be fully executed before submission.** All code cells must show

their outputs (including printed results, tables, and plots). A notebook with unexecuted or cleared cells will be treated as incomplete, and any task without visible output will receive zero credit.

3. **All plots must be displayed inline** within the notebook (not saved as separate image files).
4. **Follow the required notebook structure** (see Notebook Structure Requirement below). The grading system parses your notebook by section headers — missing or renamed headers may cause tasks to be missed during grading.
5. **Group submissions:** If you are working in a group, **every member must submit the same notebook individually on Canvas**. In addition:
 - Include a **Markdown cell at the very top** of your notebook listing all group members' names and Panther IDs.
 - Leave a **submission comment on Canvas** stating your group members (e.g., "Group: Alice Smith, Bob Lee, Carol Wang").
 - This ensures that all members of the same group are graded consistently.

Dataset

You are provided with two CSV files:

- `p2_train.csv` — Training data (~351 rows). This dataset may contain data quality issues that you need to identify and handle.
- `p2_test.csv` — Test data (80 rows). Clean held-out set for final evaluation only. **Do not use `p2_test.csv` during training or preprocessing decisions.**

The dataset uses the same columns as Project 1 (see the Dataset Description document). However, the underlying data distribution is different — the pass/fail boundary is **not linearly separable**. You should expect linear models to perform noticeably worse than nonlinear models on this data.

Notebook Structure Requirement

Your Jupyter Notebook **must** be organized with the following Markdown headers, in this exact order. This structure is required for grading purposes.

```
1 # 0. Imports and Setup
2 # 1. Data Preprocessing
3 ## 1.1 EDA and Data Cleaning
4 ## 1.2 Feature Selection and Normalization
5 # 2. Baseline: Logistic Regression (sklearn)
6 # 3. Decision Tree
```

```
7  ## 3.1 NumPy Implementation (From Scratch)
8  ## 3.2 sklearn Verification
9  ## 3.3 Overfitting Analysis
10 # 4. Support Vector Machine
11 ## 4.1 Linear SVM vs. Kernel SVM
12 ## 4.2 Hyperparameter Tuning (C and kernel)
13 # 5. Random Forest
14 ## 5.1 Training and OOB Evaluation
15 ## 5.2 Feature Importance
16 # 6. Model Comparison and Final Evaluation
17 # 7. Summary & Reflection
```

Task Descriptions

Part 1: Data Preprocessing (10 points)

Task 1.1 — EDA and Data Cleaning (5 pts: C3 + Q2)

Explore the training data and handle quality issues.

Completion (3 pts):

- Load and display basic statistics (shape, dtypes, describe)
- Identify and handle missing values, duplicates, and outliers
- Visualize the distribution of key features and the target variable

Quality (2 pts):

- Justify your cleaning decisions (why impute vs. drop, etc.)
- Discuss whether the data appears linearly separable (e.g., scatter plot of `hours_studied` vs. `sleep_hours` colored by `passed`)

Task 1.2 — Feature Selection and Normalization (5 pts: C3 + Q2)

Completion (3 pts):

- Select appropriate features with justification (should `lucky_number` be included?)
- Apply appropriate normalization/scaling
- Prepare train/test splits (or use the provided test set)

Quality (2 pts):

- Normalization parameters computed from training set only (no data leakage)
 - Clear reasoning for feature inclusion/exclusion
-

Part 2: Baseline — Logistic Regression (10 points)

Task 2.1 — Logistic Regression Baseline (10 pts: C7 + Q3)

Train a Logistic Regression model using `sklearn` as a baseline to demonstrate the limitation of linear models on this dataset.

Completion (7 pts):

- Train `LogisticRegression` on the training data
- Report training accuracy and test accuracy
- Visualize the decision boundary (use the two most important features for 2D visualization)
- Briefly explain why the linear boundary may not capture the data pattern well

Quality (3 pts):

- Compare with your Project 1 results (if applicable) — why does LR perform differently here?
 - Include a confusion matrix or classification report
 - Discuss whether the poor performance is due to underfitting or the nature of the data
-

Part 3: Decision Tree (25 points)

Task 3.1 — NumPy Implementation from Scratch (15 pts: C10 + Q5)

Implement a Decision Tree classifier from scratch using NumPy. Your implementation must include:

Completion (10 pts):

- [3] Implement a function to compute **Gini Impurity** (or Entropy) for a set of labels
- [3] Implement a function to compute **Information Gain** for a candidate split (weighted impurity reduction)
- [2] Implement the **recursive tree-building algorithm** (CART): for each node, find the best feature and threshold, split, and recurse
- [1] Implement a **predict** function that traverses the tree for a new data point
- [1] Report training accuracy on the training set

Quality (5 pts):

- [2] Code is well-structured, readable, and modular (separate functions for impurity, best split, build tree, predict)
- [1] Include at least one stopping criterion (e.g. `max_depth`, `min_samples_split`, or

- [1] Include at least one stopping criterion (e.g., `max_depth`, `min_samples_split`, or pure node)
- [1] Visualize or print the tree structure (text-based is fine, e.g., indented rules)
- [1] Test on a small toy example first to verify correctness before running on the full dataset

Task 3.2 — sklearn Verification (5 pts: C3 + Q2)

Completion (3 pts):

- Train `DecisionTreeClassifier` from sklearn with the same (or similar) hyperparameters
- Compare sklearn's training accuracy with your from-scratch implementation
- Use `sklearn.tree.plot_tree` or `export_text` to visualize the sklearn tree

Quality (2 pts):

- Discuss any differences between your implementation and sklearn's result
- Identify which feature sklearn selects as the root split — does it match yours?

Task 3.3 — Overfitting Analysis (5 pts: C3 + Q2)

Completion (3 pts):

- Train decision trees at multiple depths (e.g., 1, 2, 3, 5, 10, None)
- Report training accuracy and cross-validation accuracy for each depth
- Plot a "depth vs. accuracy" curve showing train and CV accuracy

Quality (2 pts):

- Identify the depth at which overfitting begins (train-CV gap widens)
- Connect this to the bias-variance trade-off discussed in lectures

Part 4: Support Vector Machine (20 points)

Task 4.1 — Linear SVM vs. Kernel SVM (10 pts: C7 + Q3)

Completion (7 pts):

- Train a **Linear SVM** (`SVC(kernel='linear')`) and report accuracy
- Train an **RBF kernel SVM** (`SVC(kernel='rbf')`) and report accuracy
- Visualize both decision boundaries (2D projection using the two most important features)
- Compare the two — which performs better and why?

Quality (3 pts):

- Also try `kernel='poly'` with degree 2 or 3 and compare
- Explain intuitively why the kernel SVM outperforms the linear SVM on this dataset (relate to the kernel trick and feature mapping concepts from lecture)
- Discuss what the RBF kernel is implicitly doing to the feature space

Task 4.2 — Hyperparameter Tuning (10 pts: C6 + Q4)

Completion (6 pts):

- Tune the `c` parameter for the best kernel using cross-validation (e.g., `c ∈ {0.01, 0.1, 1, 10, 100}`)
- Report CV accuracy for each C value
- Plot a "C vs. CV accuracy" curve
- Select the best C and report final test accuracy

Quality (4 pts):

- Explain the effect of C in your own words: what happens when C is very small? Very large?
 - Try tuning `gamma` for RBF as well (or use `GridSearchCV`)
 - Discuss the relationship between C and overfitting/underfitting
-

Part 5: Random Forest (20 points)

Task 5.1 — Training and OOB Evaluation (10 pts: C6 + Q4)

Completion (6 pts):

- Train a `RandomForestClassifier` with `n_estimators=100` and `oob_score=True`
- Report training accuracy, OOB accuracy, and test accuracy
- Compare Random Forest accuracy with the single Decision Tree from Part 3

Quality (4 pts):

- Explain why Random Forest outperforms a single Decision Tree (relate to bagging + feature randomization → reduced variance)
- Experiment with different `n_estimators` values (e.g., 10, 50, 100, 200, 500) and plot OOB accuracy vs. number of trees
- Discuss: does increasing `n_estimators` cause overfitting? Why or why not?

Task 5.2 — Feature Importance (10 pts: C6 + Q4)

Completion (6 pts):

- Extract and plot feature importances from the trained Random Forest
- Identify the most and least important features
- Compare with what you expected based on your EDA

Quality (4 pts):

- Discuss whether the feature importance ranking makes sense given the nonlinear patterns in the data
- Compare feature importance from Random Forest with the root split feature from your Decision Tree — are they consistent?
- Discuss any limitations of the built-in feature importance measure

Part 6: Model Comparison and Final Evaluation (no separate points — incorporated into Summary)

Use your **best model from each method** (LR, DT, SVM, RF) and evaluate all of them on the **test set**. Create a comparison table:

Model	Train Acc	CV Acc	Test Acc
Logistic Regression	...	...	...
Decision Tree (best depth)	...	...	...
SVM (best kernel + C)	...	...	...
Random Forest	...	...	...

Part 7: Summary & Reflection (15 points: C8 + Q7)

Write a summary of **400–600 words** covering:

Completion (8 pts):

- Which model performed best on this dataset, and why do you think that is?
- Why did linear models (LR, Linear SVM) struggle on this dataset compared to nonlinear models?
- What was the most surprising finding in your analysis?
- What would you try next to improve performance if you had more time?

Quality (7 pts):

Quality (7 pts).

- Demonstrate genuine understanding of the bias-variance trade-off across models
- Connect your experimental results to concepts from lecture (e.g., margin maximization, kernel trick, bagging, feature randomization)
- Provide specific evidence from your experiments (not generic statements)
- Discuss trade-offs: accuracy vs. interpretability, training time, etc.

Requirement: The summary must be written **entirely by you** (or your team members) and must demonstrate your own understanding of the project. This section will be evaluated for originality and depth of insight. Submissions where the summary does not reflect genuine engagement with the project will receive a zero for this section.

Grading Summary

Part	Task	Completion	Quality	Total
1	Data Preprocessing	6	4	10
2	Logistic Regression Baseline	7	3	10
3	Decision Tree (scratch + sklearn + overfitting)	16	9	25
4	SVM (linear vs kernel + tuning)	13	7	20
5	Random Forest (training + feature importance)	12	8	20
7	Summary & Reflection	8	7	15
	Total	62	38	100

Grading Philosophy: A student who completes every task with reasonable effort should score in the 75–85 range. Quality points reward deeper analysis, cleaner code, and stronger connections to course concepts.

Code Quality Expectations

- Code must be well-organized and readable with meaningful variable names.
- Include comments explaining your logic, especially for the from-scratch Decision Tree.
- All plots must have proper titles, axis labels, and legends.
- Use Markdown cells in your notebook to explain your approach before each code section.

Academic Integrity

You must be able to explain every line of your submitted code. Submissions that cannot be explained by the submitting student(s) during a follow-up review will receive a zero. All group members are expected to contribute meaningfully to the project.